

Statistical Computing

Monte Carlo Methods in Inference

Spring 2026

Introduction

Monte Carlo Methods for Estimation

Monte Carlo Methods for Hypothesis Tests

Application: “Count Five” Test for Equal Variance

Introduction

Monte Carlo methods: computational tools based on random sampling.

- Estimate sampling distributions, MSE, percentiles, etc.
- Assess coverage probability, Type I error rates, power of tests
- Compare statistical procedures
- Use repeated sampling from a given model (*parametric bootstrap*)

Monte Carlo Methods for Estimation

Setup

Suppose $X_1, \dots, X_n$ is a random sample from the distribution of X and let

$$\hat{\theta} = \hat{\theta}(X_1, \dots, X_n) \tag{1}$$

be an estimator of a parameter θ . Let $\mathbf{x} = (x_1, \dots, x_n)^\top \in \mathbb{R}^n$, and let $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$ denote a sequence of independent random samples generated from the distribution of X .

Random variates from the sampling distribution of $\hat{\theta}$ can be generated by repeatedly drawing $\mathbf{x}^{(j)}$ and computing $\hat{\theta}^{(j)} = \hat{\theta}(\mathbf{x}^{(j)})$.

Monte Carlo estimate of an expectation

Let $\theta = \mathbb{E}[g(X_1, \dots, X_n)]$. Based on m independent replicates,

$$\hat{\theta} = \frac{1}{m} \sum_{j=1}^m g(\mathbf{x}^{(j)}). \quad (2)$$

Example. If $X_1, X_2 \stackrel{\text{iid}}{\sim} N(0, 1)$, then $\theta = \mathbb{E} |X_1 - X_2|$ can be estimated by averaging $|x_1^{(j)} - x_2^{(j)}|$ over m independent samples.

Python: Basic Monte Carlo estimation of $\mathbb{E}|X_1 - X_2|$

```
import numpy as np

m = 1000
seed = 2026
rng = np.random.default_rng(seed)
x = rng.standard_normal((m, 2)) # m iid pairs
g = np.abs(x[:, 0] - x[:, 1]) # |X1 - X2|

theta_hat = np.mean(g)
# standard error estimate
se_hat = np.sqrt(np.sum((g - np.mean(g)) ** 2)) / m

print(theta_hat, se_hat)
```


Example output (as shown in the text)

One run gives $\hat{\theta} \approx 1.128402$.

The exact values (by integration) are

$$\mathbb{E}|X_1 - X_2| = \frac{2}{\sqrt{\pi}} = 1.128379, \quad \text{Var}(|X_1 - X_2|) = 2 - \frac{4}{\pi}, \quad (3)$$

$$se(\hat{\theta}) = \sqrt{\frac{2 - 4/\pi}{m}} = 0.02695850 \quad (m = 1000). \quad (4)$$

Estimating the standard error of the mean

For a sample $x_1, \dots, x_n$ with mean $\bar{x}$, the plug-in estimate of the variance is

$$\widehat{\text{Var}}(x) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad (5)$$

and the corresponding estimate of the standard error of $\bar{x}$ is

$$\widehat{\text{se}}(\bar{x}) = \frac{1}{\sqrt{n}} \left[\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right]^{1/2} = \frac{1}{n} \left[\sum_{i=1}^n (x_i - \bar{x})^2 \right]^{1/2}. \quad (6)$$

Using the unbiased variance estimator,

$$\widehat{\text{se}}(\bar{x}) = \frac{1}{\sqrt{n}} \left[\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \right]^{1/2}. \quad (7)$$

The mean squared error (MSE) of an estimator $\hat{\theta}$ for θ is

$$\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2]. \quad (8)$$

If m (pseudo) random samples $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$ are generated and $\hat{\theta}^{(j)} = \hat{\theta}(\mathbf{x}^{(j)})$, then the Monte Carlo estimate is

$$\widehat{\text{MSE}}(\hat{\theta}) = \frac{1}{m} \sum_{j=1}^m (\hat{\theta}^{(j)} - \theta)^2. \quad (9)$$

Trimmed mean and target MSE

Level- k trimmed sample mean

Let $X_{(1)} \leq \dots \leq X_{(n)}$ be the ordered sample. The level- k trimmed mean is

$$X_{[-k]} = \frac{1}{n - 2k} \sum_{i=k+1}^{n-k} X_{(i)}. \quad (10)$$

Assume the sampled distribution is standard normal. The target parameter is $\theta = \mathbb{E}[X] = \mathbb{E}[X_{[-1]}] = 0$. Let T denote the level-1 trimmed mean; then $\text{MSE}(T) = \mathbb{E}[T^2]$.

Python: Monte Carlo estimate of $\text{MSE}(T)$ for the level-1 trimmed mean

```
n = 20
m = 1000

tmean = np.empty(m)

for i in range(m):
    x = np.sort(rng.standard_normal(n))
    tmean[i] = np.mean(x[1:-1]) # trim 1 smallest + 1 largest

mse = np.mean(tmean ** 2)

# standard error calculation (as shown in the text)
se_hat = np.sqrt(np.sum((tmean - np.mean(tmean)) ** 2)) / m

print(mse, se_hat)
```

Example output (as shown in the text)

For the level-1 trimmed mean with $n = 20$ and $m = 1000$:

$$\widehat{\text{MSE}}(T) \approx 0.05176, \quad \widehat{se} \approx 0.00719.$$

For comparison, for the sample mean $\bar{X}$ under standard normal sampling, $\text{MSE}(\bar{X}) = \text{Var}(\bar{X}) = 1/n = 1/20 = 0.05$.

If (U, V) is a confidence interval for θ , the true confidence level is

$$\mathbb{P}(\theta \in (U, V)).$$

- Often the sampling distribution of a statistic is unknown or intractable.
- Monte Carlo: generate samples and compute the proportion $I(\theta \in C)$.

Normal-theory upper confidence limit for variance

Classical result under normal sampling

If $X_1, \dots, X_n$ is a random sample from $\text{Normal}(\mu, \sigma^2)$ ($n \geq 2$) and S^2 is the sample variance, then

$$\frac{(n-1)S^2}{\sigma^2} \sim \chi^2(n-1). \quad (11)$$

A one-sided $100(1 - \alpha)\%$ upper confidence interval for σ^2 is

$$\left(0, \frac{(n-1)S^2}{\chi_\alpha^2(n-1)} \right), \quad (12)$$

where $\chi_\alpha^2(n-1)$ is the α -quantile of $\chi^2(n-1)$.

If the sampled population is normal, then the confidence level is exactly $1 - \alpha$.

Monte Carlo experiment: empirical confidence level

Procedure

Suppose $X \sim F_X$ and θ is the target parameter. For $j = 1, \dots, m$:

1. Generate a random sample $X_1^{(j)}, \dots, X_n^{(j)}$.
2. Compute a confidence interval C_j from the j th sample.
3. Set $y_j = I(\theta \in C_j)$.

Then the empirical confidence level is $\bar{y} = \frac{1}{m} \sum_{j=1}^m y_j$.

Since $\bar{y}$ is a sample proportion, $\widehat{se}(\bar{y}) = \sqrt{\bar{y}(1 - \bar{y})/m}$.

Python: empirical confidence level for the variance UCL (normal sampling)

```
from scipy.stats import chi2

n, m = 20, 1000
alpha = 0.05

sigma = 2.0; sigma2 = sigma ** 2
q = chi2.ppf(alpha, df=n - 1)

def ucl_variance(x: np.ndarray) -> float:
    s2 = np.var(x, ddof=1) # sample variance
    return (n - 1) * s2 / q

UCL = np.array([ucl_variance(rng.normal(0.0, sigma, size=n)) for _ in range(m)])
cover = np.mean(UCL > sigma2)
se_hat = np.sqrt(cover * (1.0 - cover) / m)

print(cover, se_hat)
```

Remark: parametric bootstrap

- The problems here are *parametric* in the sense that the sampled population distribution is specified.
- This Monte Carlo approach is sometimes called the *parametric bootstrap*.

Monte Carlo Methods for Hypothesis Tests

Suppose we wish to test a hypothesis concerning a parameter θ in a parameter space Θ :

$$H_0 : \theta \in \Theta_0 \quad \text{vs} \quad H_1 : \theta \in \Theta_1, \quad (13)$$

where Θ_0 and Θ_1 partition Θ .

Two types of error:

- Type I error: reject H_0 when H_0 is true.
- Type II error: do not reject H_0 when H_0 is false.

- **Significance level** α : upper bound on $\mathbb{P}(\text{Type I error})$

$$\alpha = \sup_{\theta \in \Theta_0} \pi(\theta), \quad \text{where } \pi(\theta) = \mathbb{P}(\text{reject } H_0).$$

- **p-value**: smallest α at which the observed T^* would be significant.

Monte Carlo experiment

For each replicate $j = 1, \dots, m$:

1. Generate sample $\mathbf{x}^{(j)}$ from the null distribution.
2. Compute test statistic T_j .
3. Record $I_j = 1$ if H_0 rejected, else $I_j = 0$.

Empirical Type I error rate: $\hat{p} = \frac{1}{m} \sum_{j=1}^m I_j$.

Standard error: $\widehat{se}(\hat{p}) = \sqrt{\hat{p}(1 - \hat{p})/m}$.

Example: Empirical Type I error rate for a one-sided t -test

Suppose $X_1, \dots, X_{20}$ is a random sample from a $N(\mu, \sigma^2)$ distribution. Test

$$H_0 : \mu = 500 \quad \text{vs} \quad H_1 : \mu > 500 \quad \text{at } \alpha = 0.05. \quad (14)$$

Under the null hypothesis, $T^* = \frac{\bar{X} - 500}{S/\sqrt{20}} \sim t(19)$, where $t(19)$ denotes the Student t distribution with 19 degrees of freedom. Large values of T^* support the alternative hypothesis.

Goal: Use a Monte Carlo method to compute an empirical probability of Type I error when $\sigma = 100$, and check that it is approximately equal to $\alpha = 0.05$.

Python: Empirical Type I error rate for the one-sided t -test

```
from scipy.stats import t

n, m = 20, 10000
alpha = 0.05
mu0, sigma = 500.0, 100.0

pvals = np.empty(m)
for j in range(m):
    x = rng.normal(mu0, sigma, size=n)
    xbar, s = np.mean(x), np.std(x, ddof=1)
    tstat = (xbar - mu0) / (s / np.sqrt(n))
    pvals[j] = 1.0 - t.cdf(tstat, df=n - 1) # one-sided:  $P(T \geq tstat)$ 

p_hat = np.mean(pvals < alpha)
se_hat = np.sqrt(p_hat * (1.0 - p_hat) / m)

print(p_hat, se_hat)
```

Example output (as shown in the text)

The empirical Type I error rate and its standard error:

$$\hat{p} \approx 0.0506, \quad \widehat{se}(\hat{p}) \approx 0.00219.$$

Theoretically, the probability of rejecting the null hypothesis when $\mu = 500$ is exactly $\alpha = 0.05$ in this example.

Example: Skewness test of normality

The skewness γ_1 of a random variable X is defined by

$$\gamma_1 = \frac{\mathbb{E}[(X - \mu_X)^3]}{\sigma_X^3}, \quad \mu_X = \mathbb{E}[X], \quad \sigma_X^2 = \text{Var}(X). \quad (15)$$

- A distribution is symmetric if $\gamma_1 = 0$.
- Positively skewed if $\gamma_1 > 0$, negatively skewed if $\gamma_1 < 0$.

The sample coefficient of skewness $\hat{\gamma}_1$ is

$$\hat{\gamma}_1 = \frac{\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^3}{\left(\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2\right)^{3/2}}. \quad (16)$$

If the distribution of X is normal, then $\hat{\gamma}_1$ is asymptotically normal with mean 0 and variance $6/n$, and the test rejects for large $|\hat{\gamma}_1|$.

Under the normal limit distribution,

$$cv = z_{0.975} \sqrt{\frac{6}{n}}, \quad (17)$$

and H_0 is rejected if $|\hat{\gamma}_1| > cv$.

The Type I error rate can be assessed via Monte Carlo simulation by generating samples under H_0 (normality) and computing the proportion of rejections.

- **Type II error:** fail to reject H_0 when H_1 is true.
- **Power** $\pi(\theta)$: probability of rejecting H_0 when true parameter is θ .
- For alternative θ_1 : $\mathbb{P}(\text{Type II error}) = 1 - \pi(\theta_1)$.
- Type I error controlled by α ; high power $\Leftrightarrow$ low Type II error.

Monte Carlo experiment to estimate power

Against a fixed alternative $\theta_1 \notin \Theta_0$

1. Select a particular parameter value $\theta_1 \notin \Theta_0$.
2. For each replicate $j = 1, \dots, m$:
 - (a) Generate the j th random sample under the alternative $\theta = \theta_1$.
 - (b) Compute the test statistic T_j .
 - (c) Record the test decision: set $I_j = 1$ if H_0 is rejected at significance level α , otherwise set $I_j = 0$.
3. Compute the proportion of significant tests

$$\hat{\pi}(\theta_1) = \frac{1}{m} \sum_{j=1}^m I_j. \quad (18)$$

- To compare power across alternatives, repeat over a grid of values θ .

Example: Empirical power curve for a one-sided t -test

One-sided t -test with $n = 20$ and $\alpha = 0.05$:

$$H_0 : \mu = 500 \quad \text{vs} \quad H_1 : \mu > 500, \quad \text{with } \sigma = 100. \quad (19)$$

- Compute $\hat{\pi}(\mu)$ for a sequence of alternatives μ (e.g., $\mu = 450, 460, \dots, 650$).
- Each point on the curve is one Monte Carlo experiment.
- Estimate power by the fraction of p-values ≤ 0.05 .

The Monte Carlo standard error can be estimated by $\widehat{se}(\hat{\pi}(\mu)) = \sqrt{\hat{\pi}(\mu)(1 - \hat{\pi}(\mu)) / m}$.

Python: Empirical power curve for the one-sided t -test

```
import matplotlib.pyplot as plt

n, m = 20, 1000
alpha = 0.05
mu0, sigma = 500.0, 100.0
mu_grid = np.arange(450.0, 651.0, 10.0)

power = np.empty_like(mu_grid, dtype=float)
for i, mu1 in enumerate(mu_grid):
    # simulate under alternative mu1
    x = rng.normal(loc=mu1, scale=sigma, size=(m, n))
    xbar = x.mean(axis=1)
    s = x.std(axis=1, ddof=1)
    tstat = (xbar - mu0) / (s / np.sqrt(n))
    pvals = 1.0 - t.cdf(tstat, df=n - 1) # one-sided
    power[i] = np.mean(pvals <= alpha)
se = np.sqrt(power * (1.0 - power) / m)
```


Python: Empirical power curve for the one-sided t -test

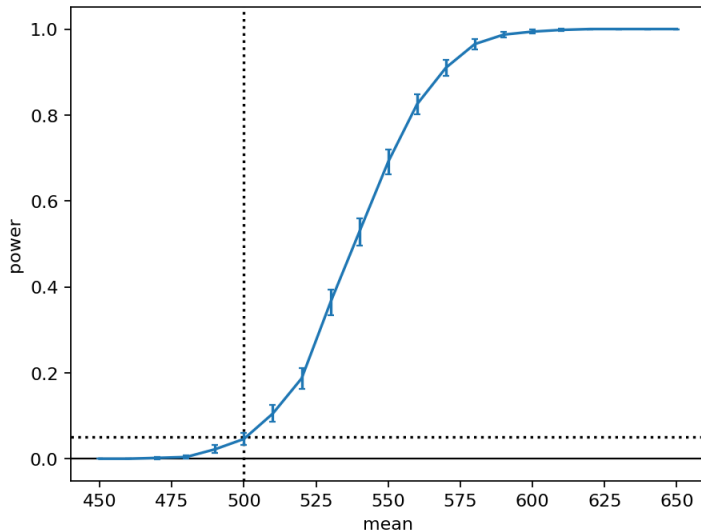
```
plt.figure()

plt.axhline(0.0, linewidth=1, c="k")
plt.axvline(mu0, linestyle=":", c="k") #  $\mu = 500$ 
plt.axhline(alpha, linestyle=":", c="k") #  $\alpha = 0.05$ 

plt.plot(mu_grid, power)
plt.errorbar(mu_grid, power, yerr=2.0 * se, fmt="none", capsize=2)

plt.xlabel("mean")
plt.ylabel("power")
plt.show()
```

Empirical power curve: figure



Application: “Count Five” Test for Equal Variance

Application: “Count Five” test for equal variance

A simple two-sample test for equal variance:

- Count *extreme points*: observations outside the range of the other sample.
- Reject H_0 (equal variance) if either sample has ≥ 5 extreme points.
- Assumes equal sample sizes and equal means.

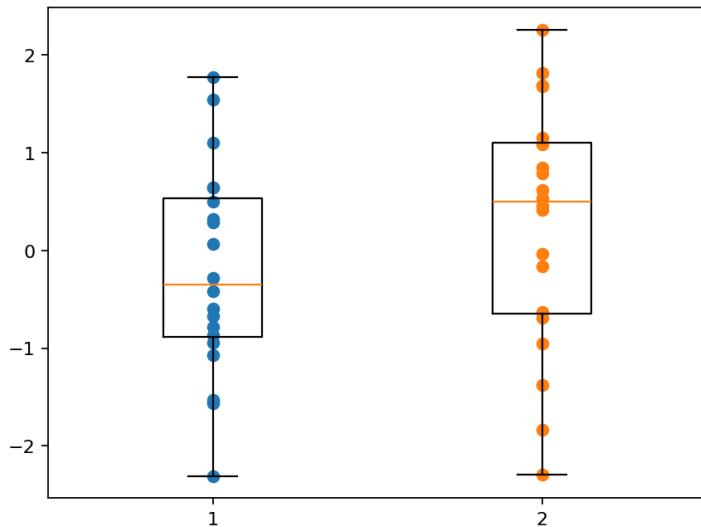
Python: Count Five statistic (numerical illustration)

```
n = 20
x1 = rng.normal(0.0, 1.0, size=n)
x2 = rng.normal(0.0, 1.5, size=n)

# side-by-side boxplots with points
plt.figure()
plt.boxplot([x1, x2], widths=0.3)
plt.scatter(np.repeat(1, n), x1)
plt.scatter(np.repeat(2, n), x2)
plt.xlim(0.5, 2.5)

plt.show()
```

Count Five statistic: figure



Python: Count Five statistic (numerical illustration)

```
# identify extreme points relative to the other sample range
```

```
i = np.where(x1 < np.min(x2))[0]
```

```
j = np.where(x2 > np.max(x1))[0]
```

```
print(x1.min(), x1.max())
```

```
print(x2.min(), x2.max())
```

```
print(x1[i])
```

```
print(x2[j])
```

```
# counts
```

```
out1 = np.sum(x1 > np.max(x2)) + np.sum(x1 < np.min(x2))
```

```
out2 = np.sum(x2 > np.max(x1)) + np.sum(x2 < np.min(x1))
```

```
print(max(out1, out2))
```

Example output (as shown in the text)

Ranges and extreme points:

$\text{range}(x_1) : (-2.783, 1.729), \quad \text{range}(x_2) : (-1.599, 3.710).$

$x_1[i] : -2.783, \quad x_2[j] : 2.036, 1.810, 3.710.$

The Count Five statistic is the maximum number of extreme points, $\max(1, 3) = 3$, so the test does not reject equal variance in this example.

Estimating the sampling distribution of the extreme-count statistic

Consider two independent random samples from the same normal distribution.

- Estimate the sampling distribution of the maximum number of extreme points.
- Find the 0.80, 0.90, and 0.95 quantiles.
- Compute the statistic by centering each sample and counting extremes relative to the range of the other centered sample.

Python: Monte Carlo estimate of the sampling distribution (normal sampling)

```
def maxout(x: np.ndarray, y: np.ndarray) -> int:
    X = x - np.mean(x)
    Y = y - np.mean(y)
    outx = np.sum(X > np.max(Y)) + np.sum(X < np.min(Y))
    outy = np.sum(Y > np.max(X)) + np.sum(Y < np.min(X))
    return int(max(outx, outy))

n1 = n2 = 20; m = 1000

stat = np.empty(m, dtype=int)
for r in range(m):
    x = rng.normal(0.0, 1.0, size=n1); y = rng.normal(0.0, 1.0, size=n2)
    stat[r] = maxout(x, y)

vals = np.arange(1, stat.max() + 1)
cdf = np.array([(stat <= v).mean() for v in vals])
q80, q90, q95 = np.quantile(stat, [0.8, 0.9, 0.95], method="higher")
```

Empirical cdf and quantiles (as shown in the text)

Empirical cdf:

stat	1	2	3	4	5	6	7	8	9	10	11
$\hat{F}$	0.149	0.512	0.748	0.871	0.945	0.974	0.986	0.990	0.996	0.999	1.000

Quantiles: 80% : 4, 90% : 5, 95% : 6.

If $\alpha = 0.05$ is desired, the critical value 5 appears to be the best choice. If a quantile function is used, compare the result to the empirical cdf.

- McGrath and Yeh: when centered by population means, significance level ≤ 0.0625 .
- In practice, center by sample mean (population mean unknown).
- Sample sizes may be unequal $\Rightarrow$ need to check Type I error.

Python: estimating the significance level (equal sample sizes)

```
n1 = n2 = 20
m = 10000

tests = np.empty(m, dtype=int)
for r in range(m):
    x = rng.normal(0.0, 1.0, size=n1); y = rng.normal(0.0, 1.0, size=n2)
    tests[r] = int(maxout(x, y) > 5)

alphahat = tests.mean()
se_hat = np.sqrt(alphahat * (1.0 - alphahat) / m)

print(alphahat)
print(se_hat)
```

Example output (as shown in the text)

Empirical Type I error rate (equal sample sizes): $\hat{\alpha} \approx 0.0565$.

- In this simulation ($m = 10000$), the Monte Carlo standard error is approximately $\sqrt{\hat{\alpha}(1 - \hat{\alpha})/m}$.
- For reference, when $\hat{\alpha} \approx 0.05$, $\sqrt{0.05 \cdot 0.95/10000} \approx 0.0022$.

When sample sizes are unequal, Type I error is **not** controlled:

- $n_1 = 20, n_2 = 30$: $\hat{\alpha} \approx 0.106$ (nominal: 0.0625)
- $n_1 = 20, n_2 = 50$: $\hat{\alpha} \approx 0.293$

⇒ Count Five test requires equal sample sizes.

Against $\sigma_1 = 1$, $\sigma_2 = 1.5$ with $n_1 = n_2 = 20$:

$$\hat{\pi} \approx 0.313 \quad (\hat{se} \approx 0.005).$$